UNISENAI - CAMPUS FLORIANÓPOLIS

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

VICTOR GABRIEL JAEGER
VINICIUS DE ASSIS MENDES
VINÍCIUS ITAMAR DE SOUZA

DESAFIO 3 – MVP Parte 1: Sistema de Medições Schulz

FLORIANÓPOLIS
2026

VICTOR GABRIEL JAEGER
VINICIUS DE ASSIS MENDES
VINÍCIUS ITAMAR DE SOUZA

DESAFIO 3 – MVP Parte 1: Sistema de Medições Schulz

Trabalho apresentado ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do UNISENAI Campus Florianópolis, como requisito parcial de avaliação da disciplina Projeto Aplicado I.

Professores: Hugo Menezes Barra
Rodrigo Losina

FLORIANÓPOLIS
2026

1. Objetivo

Desenvolver um protótipo para cadastrar técnicos e equipamentos, registrar medições e consultar seus resultados. O sistema compara o valor obtido com os limites permitidos e classifica a medição como aprovada ou reprovada.

O projeto utiliza Python, orientação a objetos e banco de dados SQLite. Os dados apresentados na demonstração são fictícios.

2. Organização do sistema

O programa foi organizado em quatro classes:

Classe	Responsabilidade
Tecnico	Representar o profissional, com matrícula, nome, e-mail e setor.
Equipamento	Representar o equipamento, com código, descrição, fabricante e modelo.
Medicao	Armazenar os valores e avaliar o resultado da medição.
BancoDados	Controlar as conexões e executar as operações no banco de dados.

As classes estão no arquivo **04_classes.py**. O arquivo **05_main.py** executa os exemplos, demonstra o CRUD e apresenta o relatório. A estrutura das tabelas está em **03_banco.sql**, e a documentação técnica está em **06_mvp_parte1.md**.

Os atributos internos utilizam sublinhado por convenção do Python. As propriedades com **@property** permitem consultar os valores sem disponibilizar um setter público. Os dados são validados durante a criação dos objetos.

3. Regra de avaliação

O intervalo permitido é calculado pelas seguintes regras:

- **Limite mínimo:** valor nominal menos a tolerância inferior.
- **Limite máximo:** valor nominal mais a tolerância superior.

A medição recebe o resultado **APROVADO** quando o valor obtido está dentro desse intervalo, incluindo os limites. Fora dele, recebe **REPROVADO**.

Na demonstração, o valor nominal foi 10,00 mm, com tolerância de 0,10 mm para cada lado. Portanto, o intervalo permitido foi de 9,90 a 10,10 mm.

Valor obtido	Intervalo permitido	Resultado
10,05 mm	9,90 a 10,10 mm	APROVADO
10,20 mm	9,90 a 10,10 mm	REPROVADO

Os cálculos utilizam **Decimal** no Python. No SQLite, os valores são armazenados em colunas **REAL**, que possuem precisão aproximada.

4. Banco de dados e tratamento de erros

O banco possui três tabelas: **tecnico**, **equipamento** e **medicao**. Cada medição está vinculada a um técnico e a um equipamento por meio de chaves estrangeiras.

A matrícula do técnico e o código do equipamento são únicos. O resultado da medição aceita somente os valores **APROVADO** ou **REPROVADO**.

O relatório utiliza **JOIN** para reunir informações das três tabelas. Os comandos SQL recebem parâmetros com **?**, evitando a concatenação dos valores diretamente nas instruções.

O programa trata dados inválidos, violações das regras do banco, falhas SQL e problemas de acesso aos arquivos. As gravações são confirmadas com **commit**. Em caso de erro SQL, **rollback** desfaz as alterações ainda não confirmadas na conexão. As conexões são fechadas no bloco **finally**.

5. Demonstração do CRUD

O CRUD completo foi implementado para as medições. Para técnicos e equipamentos, foram implementados cadastro e consulta.

A demonstração utilizou uma medição temporária para executar as quatro operações:

Operação	Resultado observado
CREATE — Criação	Medição temporária cadastrada no banco.
READ — Consulta	Registro recuperado com resultado APROVADO.
UPDATE — Atualização	Valor obtido alterado para 10,20 e resultado recalculado para REPROVADO.
DELETE — Exclusão	Medição temporária excluída pelo seu identificador.

Após a exclusão, uma nova consulta retornou **False** para a existência do registro. As duas medições anteriores foram preservadas.

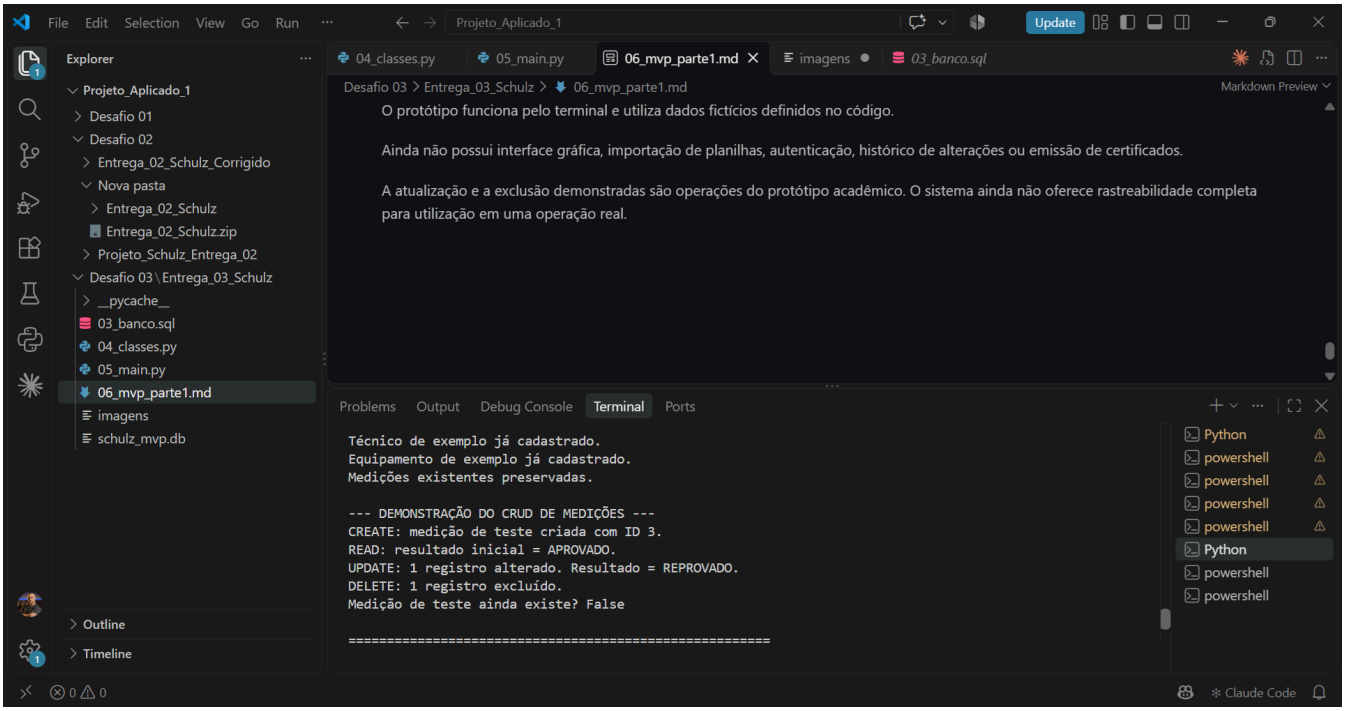


Figura 1 — Demonstração da criação, consulta, atualização e exclusão de uma medição temporária.

6. Relatório e resultados

O relatório apresenta o identificador da medição, a data registrada em UTC, o técnico responsável, o equipamento, o parâmetro avaliado, os valores e o resultado.

Na execução demonstrada, foram consultadas duas medições do equipamento **EQ001 — Paquímetro digital**, vinculadas à técnica **Ana Souza**, matrícula **TEC001**.

O resumo apresentou:

- **Total de medições:** 2.
- **Aprovadas:** 1.
- **Reprovadas:** 1.

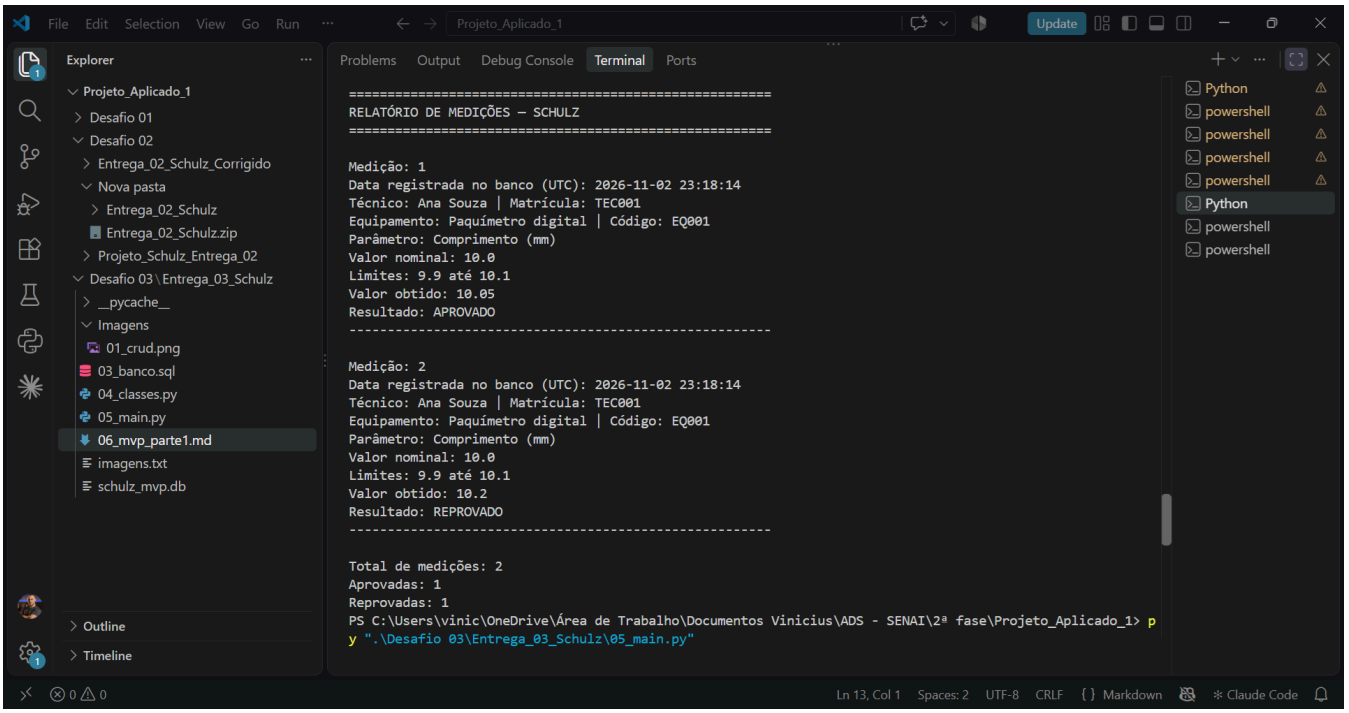


Figura 2 — Relatório das medições, relacionando os resultados ao técnico e ao equipamento.

As capturas demonstram o funcionamento do fluxo principal e do CRUD de medições. Os tratamentos de erro estão implementados, mas essas evidências não comprovam todos os cenários de falha.

7. Como executar

É necessário ter Python 3 instalado e manter o arquivo **03_banco.sql** na mesma pasta dos arquivos Python. O programa utiliza apenas bibliotecas padrão do Python.

Abra o terminal na pasta **Entrega_03_Schulz** e execute:

py 05_main.py

O programa prepara o banco e reutiliza os cadastros de exemplo existentes. Caso não haja medições, cadastra dois exemplos iniciais. Em seguida, demonstra o CRUD com um registro temporário, exclui esse registro e apresenta o relatório.

8. Limitações e conclusão

O protótipo funciona pelo terminal e utiliza dados fictícios definidos no código. Ainda não possui interface gráfica, importação de planilhas, autenticação, histórico de alterações ou emissão de certificados.

Nesta etapa, foi possível demonstrar a organização em classes, a avaliação das medições, o armazenamento no SQLite e a consulta dos resultados. O projeto oferece uma base para as próximas funcionalidades, mas ainda não possui rastreabilidade completa para utilização em uma operação real.

9. Repositório do projeto
https://github.com/Vinicius-Souza-1/projeto-schulz-entrega-03